

CITY, UNIVERSITY OF LONDON

IN3005: Computer Graphics OpenGL Coursework Report

Abhijit Shidlagatta - 210012420

Contents

1	Introduction & Architecture	2
1.1	Project Overview	2
1.2	Software Engineering & Configuration	3
2	Procedural Route and Camera Systems	4
2.1	Spline Terrain Snapping and Interpolation	4
2.2	Frenet Frames (TNB Vectors)	5
2.3	Camera Modes and Dynamic Shake	5
3	Geometry, Primitives and Audio	6
3.1	Procedural Primitives (Cylinder & Pyramid)	6
3.2	Mesh Loading and Cubemaps	6
3.3	Dynamic Lighting: Vehicle Headlights	6
3.4	Dynamic Audio System (FMOD)	7
3.5	Orthographic Text Rendering (FreeType)	7
3.6	Dynamic Particle Systems and Alpha Blending	7
3.7	Procedural Spawning and Gameplay Mechanics	8
4	Procedural Terrain and Noise	10
4.1	Procedural Heightmap Generation	10
4.2	Terrain Multitexturing and Texture Units	10
5	Advanced Rendering Techniques	12
5.1	Mathematical Frustum Culling	12
5.2	Procedural Gerstner Waves and Fresnel Reflections	12
5.3	Implementing Bloom with HDR Framebuffers	14
5.4	Directional Shadow Mapping	15
5.5	Instancing and Distance Fog	16
6	Iterative Development and Debugging	17
6.1	Iterative Post-Processing: Bloom Validation	17
6.2	Spatial Debugging Tools	17
7	Evaluation & Future Work	18
7.1	Project Strengths	18
7.2	Weaknesses & Future Expansion	18
A	Assets and References	19
A.1	3D Models and Audio Assets	19
A.2	References	19
B	Additional Screenshots and Debugging	21
C	Use of Generative AI	22

1. Introduction & Architecture

1.1 Project Overview

This project is a 3D driving game developed using OpenGL and C++. The objective was to create a procedural and non linear route that features dynamic camera systems, advanced post-processing, and driving gameplay.



Figure 1.1: A screenshot of the gameplay view in 3rd Person camera mode.

Controls and Setup

For the submission, I provided both the executable and the full source code and the visual studio .sln file where the user can easily open the project and run it. The user controls for the vehicle and camera:

Key / Input	Action / Function
W	Move vehicle forward
S	Move vehicle backward
A / D	Steer vehicle left / right
1, 2, 3	Change camera modes (1st Person, 3rd Person, Top-Down)
4	Toggle Free-Fly Debug Camera and use Mouse to rotate
F1	Play debug audio Bounce
ESC	Exit Application

Table 1.1: Table of user interface controls.

1.2 Software Engineering & Configuration

To ensure the game loop remained easy to maintain, the update sequence was divided into subsystems (`UpdatePhysicsAndInput`, `UpdateCollisions`, `UpdateParticles`). By doing this, editing and formatting the code was a much easier task. Strict attention was also paid to data typing where standard `int` types were converted to `GLuint` for all texture and buffer handles to make sure that there were no type mismatches and no memory issues. Furthermore, dynamic allocations were mainly released in class destructors to prevent memory leaks.

A core C++ architectural decision was the implementation of a centralised configuration structure at the top where variables (such as `WATER_HEIGHT`, `BEACH_TREE_LEVEL`, and collision hitboxes) were moved into this central config block. This decoupling of data from logic greatly improved scalability, and further allowed for constant iteration and tuning of the game's parameters which proved to be an improvement over hardcoding values directly into the logic functions as seen in the initial demo code.

2. Procedural Route and Camera Systems

2.1 Spline Terrain Snapping and Interpolation

The track is generated procedurally using a C^1 continuous Catmull-Rom spline as requested in the Coursework instructions (first introduced and taught in Lecture 3). The exact interpolation between four control points (p_0, p_1, p_2, p_3) is calculated mathematically to result in a smooth curvature:

Listing 2.1: Catmull-Rom Cubic Spline Interpolation

```
1 glm::vec3 CCatmullRom::Interpolate(glm::vec3& p0, glm::vec3& p1, glm::
  vec3& p2, glm::vec3& p3, float t) {
2     float t2 = t * t;
3     float t3 = t2 * t;
4
5     glm::vec3 a = p1;
6     glm::vec3 b = 0.5f * (-p0 + p2);
7     glm::vec3 c = 0.5f * (2.0f * p0 - 5.0f * p1 + 4.0f * p2 - p3);
8     glm::vec3 d = 0.5f * (-p0 + 3.0f * p1 - 3.0f * p2 + p3);
9
10    return a + b * t + c * t2 + d * t3;
11 }
```

To ensure the track rests almost perfectly on the environment, the control points listed in the config file are dynamically snapped to the geometry using the `GetTerrainHeight(x, z)` function. The track banking on the sides is achieved by calculating the cross product of the terrain's left and right slopes, tilting the spline's up-vectors to match the topography so that if the terrain slopes or the spline track is raised above the surface slightly at all, the track will follow the terrain's contour.

To make sure the vehicle maintains a consistent speed regardless of the track's curvature, the spline was re-mapped to physical distance. This makes sure that the car's velocity remains uniform, preventing it from unnaturally speeding up on straight sections or slowing down through tight corners.

2.2 Frenet Frames (TNB Vectors)

To correctly orient the car, the camera and the track geometry, a Frenet frame is calculated at every point along the spline. By deriving the Tangent (T), Normal (N), and Binormal (B) vectors, it constructs an orientation matrix that locks the vehicle to the spline track at all times. (Tangent, Normal and Binormal (TNB) frames were first introduced in Lecture 4 and 5)

2.3 Camera Modes and Dynamic Shake

Four distinct camera modes were implemented (1st Person, 3rd Person, Top-Down View and a Free-Fly Spectator mode camera). In the Free-Fly mode, the camera's orientation is calculated dynamically via mouse input. The cross product of the view vector and the up vector is calculated to ensure the camera rotates around its local axis, clamped to 1.56 radians to prevent gimbal lock which has been taught in other modules.

To show the impact of collisions between the car and different obstacles, a procedural **Camera Shake** was implemented. Rather than applying a static wobble, the shake quantity scales based on the remaining `m_collisionCooldown` timer such that the shake intensity decreases as the cooldown timer decreases.

Listing 2.2: Procedural Camera Shake with Dynamic Damping

```
1 glm::vec3 shakeOffset = glm::vec3(0.0f);
2 if (m_collisionCooldown > 0.0f) {
3     // Calculates a dynamic intensity fading smoothly from 15.0 to 0
4     float intensity = (m_collisionCooldown / 1000.0f) * 15.0f;
5
6     shakeOffset = glm::vec3(
7         ((rand() % 100) / 100.0f - 0.5f) * intensity,
8         ((rand() % 100) / 100.0f - 0.5f) * intensity,
9         ((rand() % 100) / 100.0f - 0.5f) * intensity
10    );
11 }
```

This causes a random offset into the camera's view matrix at the moment of collision/impact, which rapidly damps down to zero, creating a smooth screen shake effect.

3. Geometry, Primitives and Audio

3.1 Procedural Primitives (Cylinder & Pyramid)

To cover the coursework instructions on rendering requirements, multiple basic objects were mathematically generated from scratch in C++. The `CCylinder` class dynamically plots vertices in a circle using standard trigonometric functions ($x = r \cdot \cos(\theta)$ and $z = r \cdot \sin(\theta)$). These vertices, alongside dynamically calculated texture coordinates and surface normals, are pushed into a custom VBO wrapper classes (`CVertexBufferObject`) and rendered via `GL_TRIANGLE_STRIP` and `GL_TRIANGLE_FAN`.

The pyramid was generated by defining a square base and a single apex point, then connecting these vertices to form the faces of the pyramid. Both of these classes were designed to be reusable and easily instantiable for objects such as starting line cones and logs.

3.2 Mesh Loading and Cubemaps

More complex geometries were imported into the scene using the `COpenAssetImportMesh` (Assimp) library. To render the surrounding environment, the `CCubemap` class was used and updated (was part of the original template). This class used the `FreeImage` library to parse six high-resolution bitmap textures, mapping them to a `GL_TEXTURE_CUBE_MAP` target for use in skybox rendering and procedural reflections.

3.3 Dynamic Lighting: Vehicle Headlights

While the OpenGL template provided a general global lighting setup, the coursework required the implementation of an additional dynamic light. To achieve this, a dynamic spotlight was created and added to the front of the vehicle to act as headlights.

By extracting the forward vector (`carForward`) from the player's orientation matrix, the engine dynamically updates the spotlight's `light1.position` and `light1.direction` every frame. A sharp cutoff angle of 15.0° and an exponent of 50.0 are passed to the fragment shader, resulting in a realistic headlight that illuminates trees and obstacles dynamically as the player

moves and steers the car.

3.4 Dynamic Audio System (FMOD)

To improve the gameplay aspect as mentioned in the coursework instructions, an audio engine system was implemented using the FMOD library. The `CAudio` class manages discrete channels for ambient environments (ocean/wind) and 3D sound effects. The engine noise is dynamic where the pitch and volume of the `m_engineChannel` are linked to the absolute velocity of the vehicle, simulating RPM modulation where the faster the vehicle moves, the higher the pitch and volume.

3.5 Orthographic Text Rendering (FreeType)

The HUD uses the FreeType font library to render realtime text or data. The engine extracts the `.r` (grayscale) channel from the dynamically generated glyph textures, allowing the `textShader.frag` to overlay text on the screen using an orthographic projection matrix.

3.6 Dynamic Particle Systems and Alpha Blending

In order to make the environment feel reactive, three CPU-driven particle systems were implemented: Dirt kicks up behind the tires, leaves fall from the canopy and surround the car and wind lines streak past the camera at high speeds (which can only be seen if the player picks up the orb).

These particles are managed manually through C++ `std::vector` arrays containing custom structs. For every frame, the `UpdateParticles()` function iterates through these arrays applying basic Euler integration to update their positions based on velocity, gravity and the delta time (`m_dt`).

To render the particles correctly, control over the OpenGL state machine was required.

Listing 3.1: OpenGL State Management for Particle Blending

```
1 if (abs(m_cameraSpeed) > 0.08f) {  
2     // Enable alpha blending for transparency  
3     glEnable(GL_BLEND);  
4     glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
```

```

5
6    // Disable depth writing so particles don't block each other
7    glDepthMask(GL_FALSE);
8
9    pMainProgram->UseProgram();
10   pMainProgram->SetUniform("objectAlpha", 0.6f);
11
12   // ... Rendering loop using MatrixStack pushes and pops ...
13
14   // Restore the OpenGL state
15   glDepthMask(GL_TRUE);
16   glDisable(GL_BLEND);
17 }

```

Because transparent particles still have geometric depth, rendering them normally causes them to write to the Z-buffer which results in creating squares that block other particles behind them. By disabling using `glDepthMask(GL_FALSE)` depth-writing specifically during the particle pass, the semi-transparent geometry blends together perfectly. Each particle's transform is calculated using a `glutil::MatrixStack` by pushing and popping the matrices to make sure their local rotations and scales do not mess up the rest of the scene.

3.7 Procedural Spawning and Gameplay Mechanics

To elevate the project from a visual demo to an interactive game, procedural spawning and collision were added into the C++ game loop.

Procedural Placement

Trackside objects (such as trees, grass, barrels and logs) are spawned procedurally during initialisation. Instead of placing them randomly and manually, the engine evaluates the terrain's surface normal to ensure objects only spawn on relatively flat ground. Furthermore, a spatial check calculates the distance between the proposed spawn point and the Catmull-Rom track points, ensuring that trees and obstacles do not accidentally spawn on the drivable road.

Collisions and Pickups

The `UpdateCollisions()` subsystem checks the distance between the vehicle and obstacle hitboxes. Striking a log or barrel triggers an FMOD audio event, initiates the camera shake, sends the car backwards and activates a collision cooldown timer to prevent overlapping penalty triggers.

In addition to this, glowing Boost Orbs are procedurally placed above the track. Collecting these orbs trigger a distinct sound effect and provide a temporary increase to the vehicle's maximum speed (the speed limit enforced in the code makes sure that if the car exceeds it, it will be forced to decelerate to that set speed limit). These orbs operate on a cooldown timer, respawning mathematically after 5 seconds to provide continuous gameplay loops.

4. Procedural Terrain and Noise

4.1 Procedural Heightmap Generation

To create an organic landscape instead of a flat plane, the terrain is generated using a series of layered trigonometric functions. The engine calculates the terrain's elevation (y) by using the world-space coordinates (x, z) as inputs. By combining sine and cosine waves at different frequencies, the engine simulates naturally occurring ridges or valleys. Because the landscape is generated mathematically rather than a static 3D model, the environment stays the same every time the game is loaded, allowing for precise placement of objects like trees and the track itself. (Generating a mesh from a grid of points to form the terrain was taught in Lecture 6)

Listing 4.1: Procedural Terrain Generation using Trigonometric Noise

```
1 inline float GetTerrainHeight(float x, float z) {
2     // Exponential decay for mountain peaks
3     float dx = x + 250.0f;
4     float dz = z + 250.0f;
5     float peak = 500.0f * exp(-sqrt((dx * dx) + (dz * dz)) / 300.0f);
6
7     // Summation of sine/cosine waves for organic high-frequency noise
8     float hillWave1 = sin(x * 0.015f + z * 0.01f);
9     float hillWave2 = cos(-x * 0.01f + z * 0.015f);
10    float noise = (hillWave1 + hillWave2 + 1.5f) * 25.0f;
11
12    return peak + noise;
13 }
```

4.2 Terrain Multitexturing and Texture Units

To realistically texture the procedural terrain and cover more of the coursework requirements, a multitexturing approach was implemented rather than relying on a single stretched texture map (This advanced technique was taught in Lecture 7). Four distinct high-resolution textures (grass, rock, snow, and sand) were loaded into memory and bound to separate OpenGL texture units (GL_TEXTURE0 through GL_TEXTURE3).

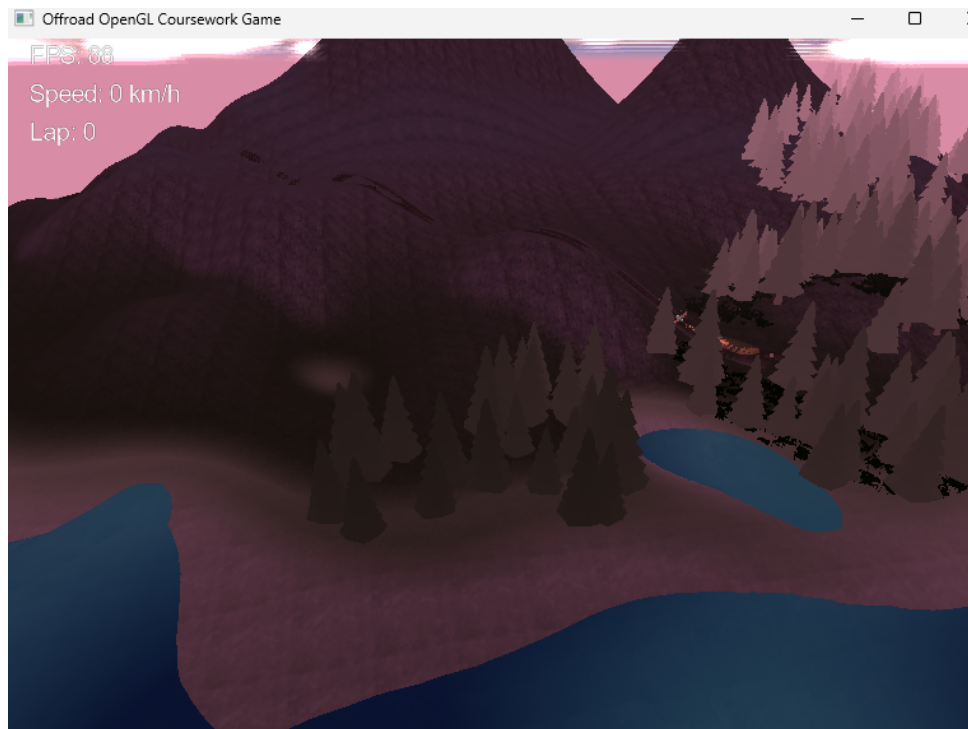


Figure 4.1: A view of the terrain demonstrating height-based multitexturing between sand, grass, and rock.

By passing all four samplers to the fragment shader simultaneously, the engine can dynamically mix them based on the terrain's height (y) and the slope's steepness (calculated via the surface normal).

Listing 4.2: Binding Multiple Texture Units in C++

```

1 pMainProgram->SetUniform("bTerrainMode", true);
2 // Binding 4 distinct texture units to the shader
3 pMainProgram->SetUniform("sampler0", 0); // Grass
4 pMainProgram->SetUniform("sampler1", 1); // Rock
5 pMainProgram->SetUniform("sampler2", 2); // Snow
6 pMainProgram->SetUniform("sampler3", 3); // Sand
7
8 m_pPlanarTerrain->Render();
9 pMainProgram->SetUniform("bTerrainMode", false);

```

This makes sure that flat areas near the water level render as sand, steep inclines render as exposed rock, and high-altitude peaks transition into snow (which wasn't included in the final implementation since the mountain's didn't reach that altitude). This satisfies the coursework requirement for multitexturing whilst also avoiding the performance downgrades of swapping texture binds mid-draw call.

5. Advanced Rendering Techniques

5.1 Mathematical Frustum Culling

To optimise CPU to GPU bandwidth, a custom Frustum Culling algorithm was implemented within the C++ game loop. Before adding a draw call for objects such as trees, barrels, logs and grass, the engine calculates the vector from the camera to the object. By taking the dot product of this vector against the camera's forward directional vector, the application determines if the object lies outside the player's current field of view.

Listing 5.1: Dot Product Frustum Culling

```
1 bool Game::IsInFrustum(const glm::vec3& objPos, const glm::vec3& cullPos
  , const glm::vec3& cullDir) {
2     glm::vec3 toObj = objPos - cullPos;
3     float dist = glm::length(toObj);
4     if (dist > 1500.0f) return false; // Far clipping plane exclusion
5
6     // Checks if the object is generally in front of the camera
7     return glm::dot(cullDir, toObj / dist) > -0.2f;
8 }
```

If the function returns false, the engine uses the `continue` statement to skip the object's matrix calculations and `glDraw` calls, resulting in an improvement in overall performance.

5.2 Procedural Gerstner Waves and Fresnel Reflections

To simulate physical ocean water, the vertex shader uses complex Gerstner Wave algorithms. Rather than simple sine or cosine wave displacement, this algorithm displaces vertices horizontally toward the crests whilst also pushing them vertically, resulting in more realistic sharp crests and wide troughs in the water surface compared to simple sine or cosine waves. The wave parameters (steepness, wavelength, speed) are all changeable via the centralised config block as mentioned already allowing for proper iterative development.

Listing 5.2: Gerstner Wave Calculation

```

1  vec3 GerstnerWave(vec2 dir, float steepness, float wavelength, float
    speed, float time, vec3 p) {
2      float k = 2.0 * 3.14159 / wavelength;
3      float c = sqrt(9.8 / k) * speed;
4      vec2 d = normalize(dir);
5      float f = k * (dot(d, p.xz) - c * time);
6      float a = steepness / k;
7
8      return vec3(d.x * (a * cos(f)), a * sin(f), d.y * (a * cos(f)));
9  }

```

In the fragment shader, the reflectiveness is determined by the Fresnel effect, which means that the water becomes highly reflective at tighter angles and highly refractive when viewed from directly above. Because the true Fresnel equations require complex and computationally expensive refractive index calculations, the engine uses **Schlick's Approximation** as researched:

$$R(\theta) = R_0 + (1 - R_0)(1 - \cos \theta)^5$$

Where R_0 is the base reflectivity of water (approximately 0.02), and θ is the angle between the view vector **V** and the surface normal **N** (calculated via the dot product).

Listing 5.3: Schlick's Fresnel Approximation in GLSL

```

1  vec3 V = normalize(-vEyePos);
2  // R0 is the base reflectivity of water
3  // R0 for water is roughly 0.02, meaning (1 - R0) is 0.98
4  float fresnel = 0.02 + 0.98 * pow(1.0 - max(dot(V, n), 0.0), 5.0);
5
6  vec3 reflectDirEye = reflect(-V, n);
7  vec3 reflectDirWorld = mat3(inverse(viewMatrix)) * reflectDirEye;
8  vec4 reflectColor = texture(CubeMapTex, reflectDirWorld);
9
10 // Blends the deep/shallow water colours with the sky reflection
11 vec3 finalColor = mix(waterBaseColor, reflectColor.rgb, fresnel);

```

This mathematical shortcut provides visually similar results to the true Fresnel equations whilst also replacing the heavy trigonometric functions with an optimised calculation. At certain angles, the water samples the environmental CubeMapTex, whereas viewing directly downwards reveals the refracted base water colours.

Depth-Based Water Colouring

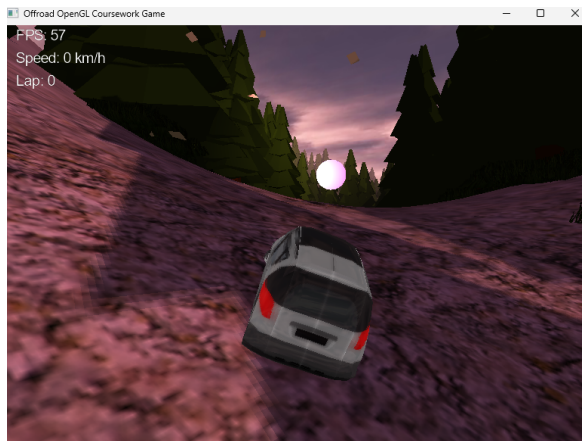
The water is not a static flat blue texture. To simulate volumetric light absorption in actual water, the fragment shader dynamically interpolates between two separate colours (deepWater and shallowWater) based on the physical height of the displaced vertex to mimic the appearance of depth.

Listing 5.4: Depth-based colour mixing in GLSL

```
1  const vec3 deepWater = vec3(0.01, 0.06, 0.18);
2  const vec3 shallowWater = vec3(0.05, 0.25, 0.35);
3
4  // Calculates a factor between 0.0 and 1.0 based on the wave's Y height
5  float heightFactor = clamp((waterHeight + 10.0) / 25.0, 0.0, 1.0);
6  vec3 waterBaseColor = mix(deepWater, shallowWater, heightFactor);
```

As the Gerstner waves push a vertex into a high crest, the shader shifts the pixel towards a lighter, translucent cyan. As the vertex drops into a deep trough, it shifts to a dark, opaque navy blue.

5.3 Implementing Bloom with HDR Framebuffers



(a) Bloom off



(b) Bloom on

Figure 5.1: A comparison demonstrating the multi-pass bloom FBO isolating and blurring the specular highlights.

To simulate bloom, a bloom pipeline was implemented using a custom Framebuffer Object (FBO) architecture as covered in Lecture 7.

Standard OpenGL framebuffers cap colour values at 1.0 (GL_UNSIGNED_BYTE). To allow

emissive elements and specular sun reflections to exceed this cap, the FBO texture formats were changed to **High Dynamic Range (HDR)** using GL_RGB16F.

Listing 5.5: HDR Framebuffer Initialisation

```
1 glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB16F, fbWidth, fbHeight, 0, GL_RGB,
  GL_FLOAT, NULL);
```

The pipeline executes over three main stages:

1. **Brightness Extraction:** The `brightFilter.frag` calculates the "Relative Luminance" of every fragment. Rather than an RGB average, I researched and implemented Rec. 709 Luma Coefficients. A threshold of 0.75 is applied so that only light sources that are strong enough and highly specular sunlight glints off the water and other reflective surfaces are passed to the next stage.
2. **Directional Blur:** The extracted bright texture is passed to `bloomComposite.frag`, which samples the image across 8 radial directions. Along each of these directions, 3 expanding samples are taken at increasing distance multipliers, which results in a soft bloom effect.
3. **Additive Compositing:** The blurred bloom effect is then added and blended back into the standard scene texture.

Listing 5.6: Biological Brightness Extraction (Rec. 709)

```
1 vec4 color = texture(sceneTexture, vTexCoord);
2 // Luminance math based on human biological light perception
3 float brightness = dot(color.rgb, vec3(0.2126, 0.7152, 0.0722));
4
5 if (brightness > 0.75) {
6     FragColor = color;
7 } else {
8     FragColor = vec4(0.0, 0.0, 0.0, 1.0);
9 }
```

5.4 Directional Shadow Mapping

Dynamic Shadow Mapping was implemented via a robust two stage pipeline (this was covered in Lecture 8). During the first stage, the scene is rendered from the sun's perspective using

an orthographic projection matrix into a high-resolution 4096x4096 Depth FBO. To eliminate self-shadowing artifacts, `glCullFace(GL_FRONT)` is temporarily enabled to exclusively render back faces into the depth map.

In the second stage, the main fragment shader transforms the fragment's position into light space. Percentage-Closer Filtering (PCF) is applied by sampling a 3×3 grid around the projected coordinate. This results in soft and more realistic shadow outlines rather than high definition jagged pixels.

Listing 5.7: Percentage-Close Filtering (PCF) for Soft Shadows

```
1 float shadow = 0.0;
2 vec2 texelSize = 1.0 / textureSize(shadowMap, 0);
3
4 for(int x = -1; x <= 1; ++x) {
5     for(int y = -1; y <= 1; ++y) {
6         float pcfDepth = texture(shadowMap, projCoords.xy + vec2(x, y) *
7             texelSize).r;
8         shadow += currentDepth - bias > pcfDepth ? 1.0 : 0.0;
9     }
10 }
11 shadow /= 9.0; // Average the 9 samples for a soft edge
```

5.5 Instancing and Distance Fog

To render complex environments without CPU overhead bottlenecks, trackside grass and forests were rendered using **Hardware Instancing** (`glDrawArraysInstanced`). A trigonometric displacement linked to the global timer t is applied strictly in the vertex shader to the top vertices of the foliage, procedurally simulating a breeze.

In addition to this, linear Distance Fog is calculated and mixed into every fragment shader to hide the clipping plane boundaries and cover the requirement from the coursework instructions (Distance Fog was explained in Lecture 8).

6. Iterative Development and Debugging

6.1 Iterative Post-Processing: Bloom Validation

Initial implementations of the Bloom thresholding algorithm used a simple $(R + G + B)/3$ arithmetic average. During testing, this resulted in dark blue and deep red objects to trigger the emissive blur. Through iterative debugging, the pipeline was updated to use the Rec. 709 Luma coefficients, making sure that only pixels perceived as physically bright by human vision triggered the FBO blur sequence.

6.2 Spatial Debugging Tools

Generating the coordinates for the Catmull-Rom track entirely through trial and error was a very slow process. To optimise this process, a spatial debugging tool was temporarily created and added to the Game Loop. By switching the camera to the Free-Fly mode (Mode 4), I was able to navigate through environment and use a raycast bound to any key to detect the exact X, Y, and Z world coordinates of the procedural terrain. This then appended and stored the valid track coordinates to a `track_config.txt` file which helped heavily in the level design process.

7. Evaluation & Future Work

7.1 Project Strengths

The core strength of the project lies in its highly scalable architecture and GPU-accelerated graphics pipeline. By offloading heavy operations to the GPU (such as Instanced Grass, Shadow Map PCF, and dynamic FBO post-processing), the CPU is able to manage gameplay logic easily. The procedural generation of the track and terrain allows for a unique environment that is not reliant on static models and the dynamic camera systems and audio engine add to the immersive gameplay experience. The use of mathematical optimisations (such as Schlick's Approximation for Fresnel reflections) demonstrates my strong understanding of both graphics programming and performance considerations as a good habit.

7.2 Weaknesses & Future Expansion

A significant limitation is the lack of accurate and rigid-body physics. Collisions are calculated using simple distance thresholds right now, which lacks proper physical simulation. To expand this project into a complete commercial simulation, integrating a dedicated physics engine (such as Bullet Physics) would be the primary focus. Additionally, implementing dynamic Level of Detail (LOD) for the procedural terrain would further optimise the pipeline by reducing the vertex count of distant geometry before submitting it to the GPU which would in turn improve performance.

A. Assets and References

A.1 3D Models and Audio Assets

As mentioned in the coursework specification, freely available third-party assets were used to populate the scene.

Asset Name	Source URL	Date Downloaded	License
Car Mesh	https://opengameart.org/content/lowpoly-car	07 April 2026	CC-BY 3.0
Tree Mesh 1	https://opengameart.org/content/low-poly-tree-1	23 April 2026	CC0
Tree Mesh 2 (Snow)	https://opengameart.org/content/low-poly-tree-1	23 April 2026	CC0
Log/Barrel Mesh	http://www.psionicgames.com/?page_id=24	24 January 2013	CC0 / Public Domain
Ocean Audio	https://pixabay.com/sound-effects/nature-ocean-waves-250310/	24 April 2026	CC0
Engine Audio	https://opengameart.org/content/engine-loop-heavy-vehicletank	01 May 2026	CC-BY 3.0
Wind Audio	https://opengameart.org/content/mild-wind-background-noise	01 May 2026	CC0
Skybox	https://opengameart.org/content/cloudy-skyboxes-0	28 April 2026	CC0
Grass Mesh	https://opengameart.org/sites/default/files/sgrass5-3.png	24 April 2026	CC0
Ground Textures	https://opengameart.org/content/wall-grass-rock-stone-wood-and-dirt-480	28 April 2026	CC0

Table A.1: External assets used within the project.

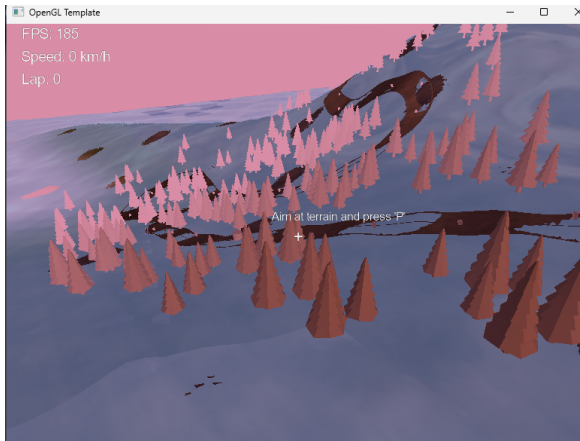
A.2 References

Benc, M. (2014). *MBSofWorks: OpenGL 3.3 and 4.0 Tutorials*. URL: <https://www.mbsoftworks.sk/index.php?page=tutorials&series=1>

- de Vries, J. (2020). *LearnOpenGL: Advanced Lighting - Bloom*. URL: <https://learnopengl.com/Advanced-Lighting/Bloom>
- de Vries, J. (2020). *LearnOpenGL: Comprehensive Graphics Programming*. URL: <https://learnopengl.com/>
- Finch, M. (2004). *Effective Water Simulation from Physical Models*. GPU Gems. NVIDIA. URL: <https://developer.nvidia.com/gpugems/gpugems/part-i-natural-effects/chapter-1-effective-water-simulation-physical-models>
- International Telecommunication Union (2015). *Recommendation ITU-R BT.709-6: Parameter values for the HDTV standards for production and international programme exchange*. URL: <https://www.itu.int/rec/R-REC-BT.709/en>
- Khronos Group (2026). *docs.gl: OpenGL API Documentation*. URL: <https://docs.gl/>
- OpenGameArt (2026). *OpenGameArt.org - Free Texture and Audio Assets*. URL: <https://opengameart.org/>
- opengl-tutorial.org (2026). *Tutorials for OpenGL 3.3 and later*. URL: <https://www.opengl-tutorial.org/>
- Schlick, C. (1994). *An Inexpensive BRDF Model for Physically-Based Rendering*. Computer Graphics Forum, 13(3), pp. 233-246.
- Swiftless (2026). *OpenGL 4 Tutorials*. URL: <https://www.swiftless.com/opengl4tuts.html>
- Wikipedia (2026). *Centripetal Catmull-Rom spline*. URL: https://en.wikipedia.org/wiki/Centripetal_Catmull-Rom_spline

B. Additional Screenshots and Debugging

Throughout the development of this engine, numerous graphical artifacts and rendering bugs were encountered. Documenting these failures was a crucial part of the iterative development process.



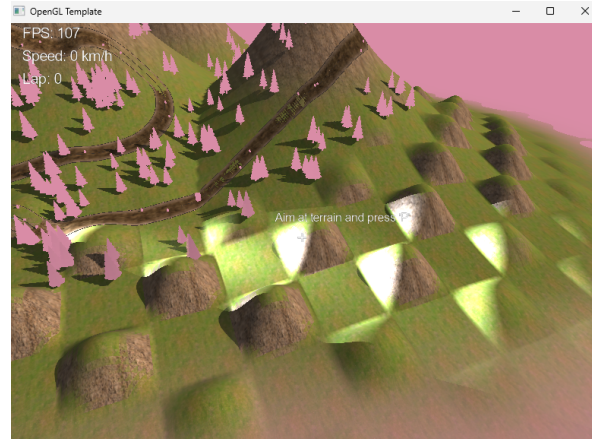
(a)



(b)



(c)



(d)

Figure B.1: Visual debugging and graphical artifacts encountered during development.

C. Use of Generative AI

During the development of this project, I used Google's Gemini AI as a pair-programming assistant. As stated in my demo submission declaration, my primary use cases were optimising existing C++ architectures, understanding advanced rendering mathematics, troubleshooting complex graphical artifacts in my GLSL shaders (specifically fixing the brightness of the bloom, and the colouring of the distance fog) and as a search tool for finding relevant documentation and examples.

I provided my existing code and screenshots of visual bugs to the AI and used its output to understand the underlying mathematics and correct my implementations. However, I did not blindly copy and paste generated content. All generative math concepts were reviewed against my lecture and lab notes to reinforce my knowledge before I manually integrated, tuned, and adapted the logic into my specific C++ and OpenGL engine. Below are specific examples of the sequences of prompts used and the outputs received, fulfilling the coursework requirements for generative AI usage:

Example 1: Mathematical Rendering (HDR Bloom)

Explanation: I asked the AI to explain why standard colour averaging resulted in poor visual bloom, triggering on dark objects.

Prompt: "my bloom effects look weird and trigger on dark blue and red objects when I just average the RGB values like $(R+G+B)/3$?"

Output: "Averaging the RGB values mathematically treats all colours equally, but human eyes do not perceive light that way. We are much more sensitive to green than to blue. If you have a dark, deep blue object, an arithmetic average might still push it above your bloom threshold. Instead of an average, you should use relative luminance (Rec. 709 coefficients). You can calculate this in GLSL using a dot product with biological light weights: `float brightness = dot(color.rgb, vec3(0.2126, 0.7152, 0.0722));`. This ensures only pixels that actually *look* bright to the human eye trigger the bloom FBO."

Example 2: Shader Debugging (Procedural Water)

Explanation: I used the AI to help troubleshoot an issue where my procedural water shader was applying wave math to the entire environment instead of just the water. This can be seen in the additional screenshots section.

Prompt: "My water shaders are applying the waves to my whole terrain floor instead of just the flat water plane and both the terrain and water floor are moving up and down."

Output: "This is a classic OpenGL state machine issue! You are likely binding your `waterShader` program, drawing your water plane, and then forgetting to switch back to your `mainShader` before drawing the terrain. Because OpenGL is a state machine, it keeps using the water vertex shader for the terrain geometry. To fix this, ensure you call `pMainProgram->UseProgram()`; immediately after your water rendering block, right before you call `m_pPlanarTerrain->Render();`."

Example 3: Logic and Animation Errors (Foliage)

Explanation: I needed help identifying why my instanced tree wind animation was scaling with the vehicle's movement speed rather than acting independently in the game world.

Prompt: "My instanced trees are waving in the wind way too fast when I drive the car forward. When I stop the car, the trees stop waving completely. Why is the car's velocity affecting my vertex shader animation?"

Output: "Look at how you are updating the `t` (time) uniform being passed to your foliage shader in C++. If the trees stop moving when the car stops, you are likely passing the car's accumulated position or velocity into the time uniform instead of the global application time. Find where you call `SetUniform("t", ...)` for the foliage. Change it from using the car's movement variable to your global elapsed time variable or your delta time (`dt`) accumulator."

Example 4: Demo Polish (Fog and Lighting)

Explanation: During the final stages of demo preparation, I used the AI to refine my post-processing effects, specifically fixing the distance fog colour mixing.

Prompt: "my distance fog looks like a flat grey wall instead of blending naturally, and my bloom

is washing out the screen luckily the logic ive writetn works in it so dont try to ruin it but how do i fix the colour mixing nad the bloom?"

Output: "For the fog, instead of just adding the fog colour, you need to use GLSL's `mix()` function based on the fragment's distance from the camera. Create a fog factor between 0.0 and 1.0, and calculate it like this: `FragColor = mix(originalColor, fogColor, fogFactor);`. For the bloom washing out the screen, your extraction threshold is too low. In your bright-filter pass, raise the threshold from 0.5 to 0.85 or 0.9 so that only the actual sun and direct specular reflections are captured by the blur framebuffer."

Example 5: Document Formatting (LaTeX Template)

Explanation: I used the AI to generate a LaTeX template that mirrored the style of my previous dissertation. I chose to write my report in LaTeX because it is the industry standard for accurately and professionally displaying C++ code blocks, complex mathematical algorithms, and side-by-side graphical comparisons.

Prompt: "Can you write a LaTeX document template for me that looks like this latex template i used for my disssertation here? I need it to cleanly format C++ code blocks, GLSL shaders, math equations, and side-by-side images."

Output: "Here is a complete `main.tex` template. I have included the `listings` package for your C++ and GLSL code with custom syntax highlighting, the `amsmath` package for your Fresnel and Gerstner wave equations, and the `subcaption` package for your side-by-side bloom comparison images. I have also formatted the chapters and sections to match standard academic guidelines."

Example 6: Inline Syntax Autocomplete (Visual Studio)

Explanation: In addition to Gemini, I left Visual Studio's built-in AI autocomplete Copilot enabled during development. This tool was not used to generate new logic or engine architecture, but instead acted as a minor assistant to save time on boilerplate typing. It was mainly used to predict the comments for my code.

Prompt: *(No explicit prompt. The AI reads the current working file context and suggests inline*

ghost text).

Output: The engine would predict the remainder of a standard OpenGL function call or a comment based on the small context of its surrounding code. Because the AI lacks wider architectural context, its suggestions to do with code generation were constantly incorrect regarding game logic, meaning its usage was strictly limited to saving time on standard syntax but was generally quite useful in terms of predicting code and providing assistance with commenting and boosting productivity.

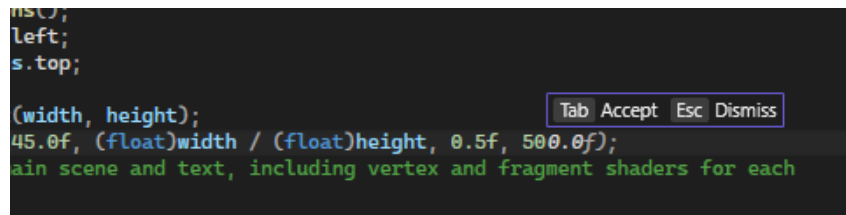


Figure C.1: An example of Visual Studio's built-in AI predicting the completion of an OpenGL projection matrix line, requiring me to press 'Tab' to verify and accept the change suggested.